Name : Vivek Singh
RegNumber : 230905408
Roll Number: A50

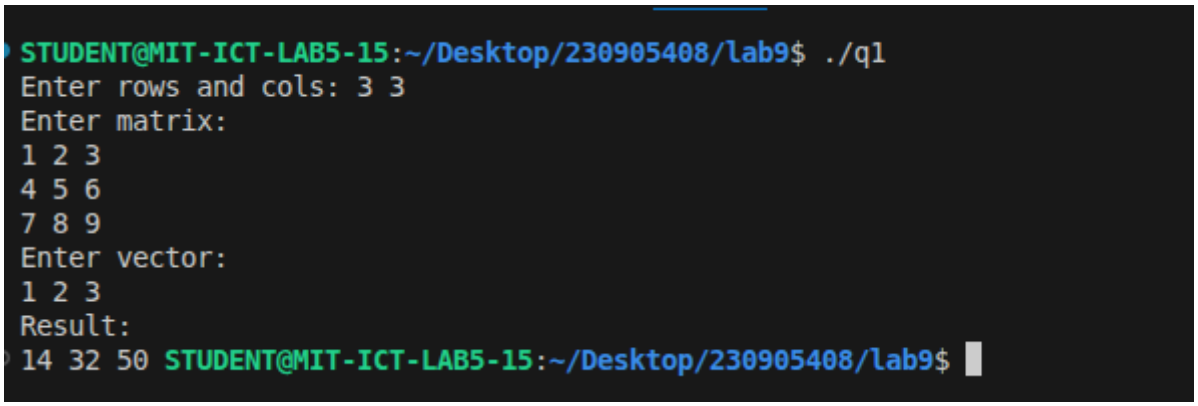
Lab 8: Programs on Matrix in CUDA

Q1).
Code:

```
#include <stdio.h>
#include <cuda.h>

__global__ void spmv(int *data, int *col_index, int *row_ptr, int *x, int *y, int rows) {
int row = threadIdx.x + blockIdx.x * blockDim.x;

if (row < rows) {
int sum = 0;
for (int j = row_ptr[row]; j < row_ptr[row + 1]; j++) {
sum += data[j] * x[col_index[j]];
}
y[row] = sum;
}
}

int main() {
int rows, cols;

printf("Enter rows and cols: ");
scanf("%d %d", &rows, &cols);

int A[100][100], x[100];

printf("Enter matrix:\n");
for (int i = 0; i < rows; i++)
for (int j = 0; j < cols; j++)
scanf("%d", &A[i][j]);

printf("Enter vector:\n");
for (int i = 0; i < cols; i++)
scanf("%d", &x[i]);

// Convert to CSR
int data[100], col_index[100], row_ptr[100];
int k = 0;

row_ptr[0] = 0;
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
if (A[i][j] != 0) {
```

```
data[k] = A[i][j];
col_index[k] = j;
k++;
}
}
row_ptr[i + 1] = k;
}

int *d_data, *d_col, *d_row, *d_x, *d_y;
int y[100];

cudaMalloc(&d_data, k * sizeof(int));
cudaMalloc(&d_col, k * sizeof(int));
cudaMalloc(&d_row, (rows + 1) * sizeof(int));
cudaMalloc(&d_x, cols * sizeof(int));
cudaMalloc(&d_y, rows * sizeof(int));

cudaMemcpy(d_data, data, k * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_col, col_index, k * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_row, row_ptr, (rows + 1) * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_x, x, cols * sizeof(int), cudaMemcpyHostToDevice);

spmv<<<1, rows>>>(d_data, d_col, d_row, d_x, d_y, rows);
cudaMemcpy(y, d_y, rows * sizeof(int), cudaMemcpyDeviceToHost);

printf("Result:\n");
for (int i = 0; i < rows; i++)
printf("%d ", y[i]);

return 0;
}
```

Output :


STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab9$ ./q1
Enter rows and cols: 3 3
Enter matrix:
1 2 3
4 5 6
7 8 9
Enter vector:
1 2 3
Result:
14 32 50 STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab9$

Q2)

Code:

```
#include <stdio.h>
#include <cuda.h>

__global__ void transform(int *A, int *B, int rows, int cols) {
int idx = threadIdx.x + blockIdx.x * blockDim.x;

if (idx < rows * cols) {
int row = idx / cols;
int val = A[idx];

if (row == 1)
B[idx] = val * val;
else if (row == 2)
B[idx] = val * val * val;
else
B[idx] = val;
}
}

int main() {
int rows = 3, cols = 3;

int h_A[] = {
1,2,3,
4,5,6,
7,8,9
};

int h_B[9];

int *d_A, *d_B;

cudaMalloc(&d_A, 9 * sizeof(int));
cudaMalloc(&d_B, 9 * sizeof(int));

cudaMemcpy(d_A, h_A, 9 * sizeof(int), cudaMemcpyHostToDevice);

transform<<<1, 9>>>(d_A, d_B, rows, cols);
cudaMemcpy(h_B, d_B, 9 * sizeof(int), cudaMemcpyDeviceToHost);
printf("Output:\n");
for (int i = 0; i < 9; i++) {
printf("%d ", h_B[i]);
if ((i+1)%cols==0) printf("\n");
}

return 0;
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab9$ ./q2
Output:
1 2 3
16 25 36
343 512 729
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905408/lab9$
```

Q3)

```
// border_replace.cu
#include <stdio.h>
#include <cuda.h>

__global__ void process(int *A, int *B, int rows, int cols) {
int idx = threadIdx.x + blockIdx.x * blockDim.x;

if (idx < rows * cols) {
int r = idx / cols;
int c = idx % cols;

if (r == 0 || r == rows - 1 || c == 0 || c == cols - 1)
B[idx] = A[idx]; // border
else
B[idx] = ~A[idx]; // 1's complement
}
}

int main() {
int rows = 4, cols = 4;

int h_A[] = {
1,2,3,4,
6,5,8,3,
2,4,10,1,
9,1,2,5
};

int h_B[16];

int *d_A, *d_B;

cudaMalloc(&d_A, 16 * sizeof(int));
cudaMalloc(&d_B, 16 * sizeof(int));

cudaMemcpy(d_A, h_A, 16 * sizeof(int), cudaMemcpyHostToDevice);

process<<<1, 16>>>(d_A, d_B, rows, cols);
```

```
cudaMemcpy(h_B, d_B, 16 * sizeof(int), cudaMemcpyDeviceToHost);

printf("Output:\n");
for (int i = 0; i < 16; i++) {
printf("%d ", h_B[i]);
if ((i+1)%cols==0) printf("\n");
}

return 0;
}
```

OUTPUT: